R codes for the NutNet-Stability project

Step4.2: Dimensionality analysis

Qiang Yang

2026-06-22

Overview

This workflow evaluates whether the dimensionality of ecological stability differs between plots experiencing relatively low and high nutrient enrichment intensities. Dimensionality is quantified as the relative length of the first semi-axis of the multidimensional ellipsoid derived from the covariance matrix of four stability components: functional temporal variability, extent of functional change, compositional temporal variability, and extent of compositional change. This measure is mathematically equivalent to the proportion of variance explained by the first principal component and provides an integrated measure of the strength of relationships among stability responses.

Because dimensionality describes covariance structures among multiple plots, it cannot be estimated at the level of individual plots. Plots are therefore classified into low- and high-enrichment groups according to whether their relative enrichment intensities are below or above the median enrichment intensity within each nutrient treatment.

Preliminary analyses demonstrated that neither temporal window length nor its interaction with enrichment intensity affected stability dimensionality. Consequently, this workflow focuses on the effects of enrichment intensity while accounting for the temporal autocorrelation that arises because dimensionality estimates are repeatedly calculated from progressively longer temporal windows.

The workflow first calculates the dimensionality of stability for each nutrient treatment, enrichment-intensity group, and temporal window. Generalized least-squares (GLS) models are then used to test whether dimensionality differs between low- and high-enrichment groups while accounting for temporal autocorrelation among repeated dimensionality estimates obtained from progressively longer temporal windows.

Statistical significance is assessed using permutation tests. For each permutation, enrichment-intensity group assignments are randomly permuted among plots while preserving the temporal structure of each plot. The GLS coefficient representing the difference between enrichment groups is recalculated repeatedly to generate a null distribution against which the observed coefficient is compared.

The complete workflow typically requires approximately 15 minutes to run on a standard desktop computer. Computational time can be reduced by decreasing the number of permutation iterations at Line 167 (e.g., to 100), although this may reduce the precision of the estimated null distributions and associated significance tests.

Output

Running this workflow generates:

1. Figures showing differences in the dimensionality of ecological stability between low- and high-enrichment intensity groups across the three nutrient treatments (main Fig. 4a and Fig. S8a).
2. Figures showing how pairwise relationships among the four stability components differ between low- and high-enrichment intensity groups across nutrient treatments (main Fig. 4b and Fig. S8b).

Section 1: setup work environment

```
rm(list = ls())
gc()
```

```
##          used (Mb) gc trigger (Mb) max used (Mb)
## Ncells 496707 26.6    1075957 57.5   686411 36.7
## Vcells 929932  7.1    8388608 64.0  1876579 14.4
```

```
# necessary R packages
Packages <- c(
  "tidyverse", "magrittr", "grid", "gridExtra", "scales",
  "broom", "dlookr", "nlme", "patchwork", "cowplot"
)
pacman::p_load(char = Packages)
# functions with conflicting status (presence in different packages used here)
select <- dplyr::select
summarise <- dplyr::summarise
# working path
main.path <- getwd()
data.path <- paste0(main.path, "/mydata")
figure.path <- paste0(main.path, "/myfigure")
set.seed(1234)
```

Section 2: dimensionality analysis using the PCA approach (func_var_det used)

Detrended functional temporal variability (func_var_det) is used throughout this workflow because it was employed in the manuscript analyses. The non-detrended functional variability metric is not considered further here.

2.1 data organization - observed data

Note that all stability components here are first scaled so that they have the similar scale.

```
load(paste0(data.path, "/df.tidy.stabilities.Rdata"))
load(paste0(data.path, "/df.tidy.Rdata"))
df.full <- df.tidy.stabilities %>%
```

```
  left_join(df.tidy %>% select(-contains("data")))
rm(df.tidy, df.tidy.stabilities)

df.full %<>%
  select(site_code:end_year, matches("_added_proportion|change|var"))
sta.names <- colnames(df.full) %>% str_subset("change|var")

# For the PCA analysis, all stability components need to be available for each row
# no NA, no infinite
# remove the relevant cases
df.full <- df.full %>%
  filter_all(all_vars(!is.infinite(.))) %>%
  filter_all(all_vars(!is.na(.)))

df.stability <- df.full

# scale stability components
df.stability %<>% mutate(across(one_of(sta.names), scale))


# make a meta data for multidimensionality analysis
trt.set <- c("N", "P", "K")
final.year.set <- 9 # original using 5:9
df.meta <- crossing(trt.set, final.year.set)

## add data, nested by year

df.meta <- df.meta %>%
  mutate(data = map2(
    trt.set, final.year.set,
    function(x1, x2) {
      plots.to.keep <- df.stability %>%
        filter(
          trt == x1,
          end_year == x2
        ) %>%
        select(site_code, block, trt, plot)
      subdat <- df.stability %>%
```

```
        semi_join(plots.to.keep) %>%
        filter(
          trt == x1,
          end_year <= x2
        )
      return(subdat)
    }
  ))
df.meta %<>% ungroup()

## add data, nested by perturbation intensity and end_year

df.meta <- df.meta %>%
  mutate(data2 = map2(
    trt.set, data,
    function(x1, dat) {
      df.pert <- dat %>%
        select(site_code, block, trt, plot, paste0(x1, "_added_proportion")) %>%
        distinct() %>%
        set_colnames(c("site_code", "block", "trt", "plot", "PI"))
      df.pert %<>%
        mutate(PI.group = ifelse(PI > median(PI), "High", "Low"))
      dat %<>% left_join(df.pert)
      dat.nested <- dat %>%
        group_by(end_year, PI.group) %>%
        nest() %>%
        ungroup()
      return(dat.nested)
    }
  ))

df.meta %<>%
  select(-data) %>%
  unnest(cols = data2)

df.N.P.K <- df.meta %>% ungroup()

# # further only select stability components from data
```

```
df.N.P.K %<>%
  mutate(data = map(data, function(dat) dat %>% select(one_of(sta.names))))
df.N.P.K.obs <- df.N.P.K
```

2.2 data organization - data produced by permutation

Conduct permutations. The dataset full were permuted by plot*final.year.set*end_year.

```
set.seed(1234)
L <- list()
for (i in 1:1000) {
  # monitor the progress of the permutation procedure
  if(i %% 50 == 0) {
    print(paste("Permutation", i, "out of 1000"))
    print(Sys.time())
  }
  df.stability.obs <- df.stability
# first randomize the nutrient perturbaiton intensity of plots
df.nutrient.pert <- df.stability.obs %>%
  select(site_code, block, trt, plot, contains("proportion")) %>%
  distinct()# the nutrient perturbation of each plot
df.nutrient.pert.random <- df.nutrient.pert %>%
  group_by(trt) %>%
  nest() %>%
  mutate(data2 = map(data, function(dt_){
    dt_ %>%
      mutate(N_added_proportion = sample(N_added_proportion),
             P_added_proportion = sample(P_added_proportion),
             K_added_proportion = sample(K_added_proportion))
  })) %>%
  select(-data) %>%
  unnest(cols = data2)
df.stability.random.plots <- df.stability.obs %>%
  select(-contains("proportion")) %>%
  left_join(df.nutrient.pert.random)

df.stability.0 <- df.stability.random.plots
rm(df.stability.random.plots)
```

```r
# make a meta data for multidimensionality analysis
trt.set <- c("N", "P", "K")
final.year.set <- 9 # original using 5:9
df.meta <- crossing(trt.set, final.year.set)

## add data, nested by year

df.meta <- df.meta %>%
  mutate(data = map2(
    trt.set, final.year.set,
    function(x1, x2) {
      plots.to.keep <- df.stability.0 %>%
        filter(
          trt == x1,
          end_year == x2
        ) %>%
        select(site_code, block, trt, plot)
      subdat <- df.stability.0 %>%
        semi_join(plots.to.keep) %>%
        filter(
          trt == x1,
          end_year <= x2
        )
      return(subdat)
    }
  ))
df.meta %<>% ungroup()

## add data, nested by perturbation intensity and end_year

df.meta <- df.meta %>%
  mutate(data2 = map2(
    trt.set, data,
    function(x1, dat) {
      df.pert <- dat %>%
        select(site_code, block, trt, plot, paste0(x1, "_added_proportion")) %>%
        distinct() %>%
        set_colnames(c("site_code", "block", "trt", "plot", "PI"))
```

```
        df.pert %<>%
          mutate(PI.group = ifelse(PI > median(PI), "High", "Low"))
        dat %<>% left_join(df.pert)
        dat.nested <- dat %>%
          group_by(end_year, PI.group) %>%
          nest() %>%
          ungroup()
        return(dat.nested)
      }
    ))

  df.meta %<>%
    select(-data) %>%
    unnest(cols = data2)

  df.N.P.K <- df.meta %>% ungroup()

  # # further only select stability components from data
  df.N.P.K %<>%
    mutate(data = map(data, function(dat) dat %>% select(one_of(sta.names))))

  L[[i]] <- df.N.P.K
  rm(df.N.P.K)
}
```

```
## [1] "Permutation 50 out of 1000"
## [1] "2026-06-22 13:53:11 CEST"

## [1] "Permutation 100 out of 1000"
## [1] "2026-06-22 13:53:28 CEST"

## [1] "Permutation 150 out of 1000"
## [1] "2026-06-22 13:53:44 CEST"

## [1] "Permutation 200 out of 1000"
## [1] "2026-06-22 13:53:59 CEST"

## [1] "Permutation 250 out of 1000"
## [1] "2026-06-22 13:54:14 CEST"
```

```
## [1] "Permutation 300 out of 1000"
## [1] "2026-06-22 13:54:30 CEST"

## [1] "Permutation 350 out of 1000"
## [1] "2026-06-22 13:54:49 CEST"

## [1] "Permutation 400 out of 1000"
## [1] "2026-06-22 13:55:06 CEST"

## [1] "Permutation 450 out of 1000"
## [1] "2026-06-22 13:55:25 CEST"

## [1] "Permutation 500 out of 1000"
## [1] "2026-06-22 13:55:41 CEST"

## [1] "Permutation 550 out of 1000"
## [1] "2026-06-22 13:55:58 CEST"

## [1] "Permutation 600 out of 1000"
## [1] "2026-06-22 13:56:14 CEST"

## [1] "Permutation 650 out of 1000"
## [1] "2026-06-22 13:56:29 CEST"

## [1] "Permutation 700 out of 1000"
## [1] "2026-06-22 13:56:45 CEST"

## [1] "Permutation 750 out of 1000"
## [1] "2026-06-22 13:57:01 CEST"

## [1] "Permutation 800 out of 1000"
## [1] "2026-06-22 13:57:16 CEST"

## [1] "Permutation 850 out of 1000"
## [1] "2026-06-22 13:57:33 CEST"
```

```
## [1] "Permutation 900 out of 1000"
## [1] "2026-06-22 13:57:50 CEST"

## [1] "Permutation 950 out of 1000"
## [1] "2026-06-22 13:58:06 CEST"

## [1] "Permutation 1000 out of 1000"
## [1] "2026-06-22 13:58:23 CEST"
```

```r
names(L) <- 1:length(L)
df.N.P.K.perm <- tibble(
  perm.id = names(L), # This will be the ID column
  data = L # This will be the column containing data.frames
) %>%
  unnest(data) %>%
  mutate(perm.id = as.numeric(perm.id))

df.permutation <- df.N.P.K.obs %>%
  mutate(
    data.group = "obs",
    perm.id = 0
  ) %>%
  bind_rows(df.N.P.K.perm %>%
    mutate(data.group = "permutation"))
```

2.3 compute the covariance matrix and the eigen vector of the covariance matrix

```r
df.permutation <- df.permutation %>%
  mutate(data = map(data, function(dt1) dt1 %>% select(-func_var)))
# calculate covariance matrix and eigenvector
df.permutation %<>% mutate(cov.M = map(data, cov))
df.permutation %<>% mutate(eigvec = map(cov.M, function(x) eigen(x)$vectors))
#### calculate the variance explained by first eigen vector of the covariance matrix
df.permutation %<>%
  mutate(var.explained = map2_dbl(data, eigvec, function(D, X) {
    D <- as.matrix(D)
    P <- D %*% X
    V <- apply(P, 2, var)
```

```
    V[1] / sum(V)
  }))

df.variance.explained <- df.permutation %>%
  select(trt.set, final.year.set, end_year, PI.group, data.group, perm.id, var.explained)
```

2.4 GLS for the observed and the permutated

```
# i first nest the df.variance explained, so that each row can have one gls
df.nested <- df.variance.explained %>%
  select(-final.year.set) %>% # unique value, so can be removed
  group_by(trt.set, perm.id, data.group) %>%
  nest()
# perform the gls
df.nested <- df.nested %>%
  mutate(model.results = map(data, function(dt_) {
    gls_model <- gls(var.explained ~ PI.group,
      data = dt_,
      correlation = corAR1(form = ~ end_year | PI.group)
    )
  }))
# extract the coefficient representing the difference in semi-axis length
df.nested <- df.nested %>%
  mutate(diff_value = map_dbl(model.results, function(model_) coef(model_)["PI.groupLow"]))

# note that the difference now is calculated as low-perturbation minus high-perturbation,
# I want to use it in an opposite way
df.nested <- df.nested %>%
  mutate(diff_value = -1 * diff_value)
# now factorize trt.set
df.nested <- df.nested %>%
  mutate(trt.set = paste0("+", trt.set)) %>%
  mutate(trt.set = as.factor(trt.set)) %>%
  mutate(trt.set = fct_relevel(trt.set, c("+N", "+P", "+K")))

# only keep the key columns
df.results <- df.nested %>%
  select(-data, -model.results) %>%
```

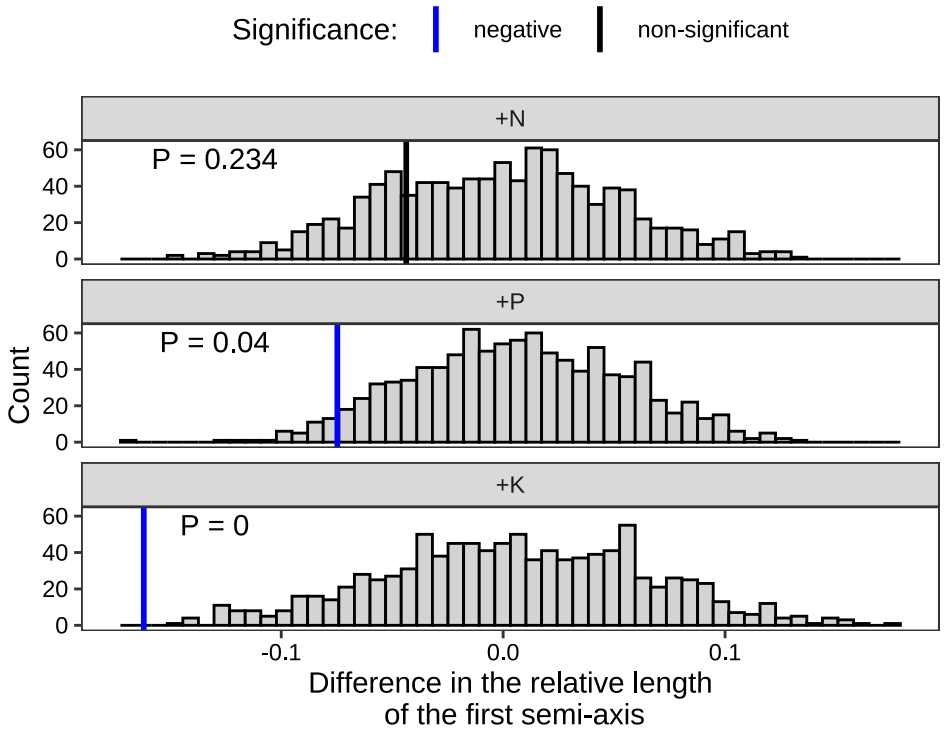
```
  group_by(trt.set) %>%
  nest()
# now add a column to represent the significance
df.results <- df.results %>%
  mutate(alpha = map_dbl(data, function(dt_) {
    x <- dt_ %>%
      filter(perm.id != 0) %>%
      select(diff_value) %>%
      unlist()
    x0 <- dt_ %>%
      filter(perm.id == 0) %>%
      select(diff_value) %>%
      unlist()
    length(which(x <= x0)) / length(x)
  }))
# also add a denote colomn for p values so I can add it to the figure later
df.results <- df.results %>%
  mutate(denote = map_chr(alpha, function(x) {
    paste("P =", x)
  }))
# add a column represent significance groups
df.results <- df.results %>%
  mutate(sig.group = map_chr(alpha, function(x) {
    if (x < 0.05) {
      y <- "negative"
    } else if (x < 0.95) {
      y <- "non-significant"
    } else {
      y <- "positive"
    }
  }))
df.nested.decoupling.var.explained_func_var_det <- df.nested
df.results.decoupling.var.explained_func_var_det <- df.results
```

2.5 Histogram of the GLS model coefficients

```
p.null <- df.nested %>%
  filter(data.group == "permutation") %>%
```

```
  ggplot(aes(x = diff_value)) +
  facet_wrap(~trt.set, nrow = 3) +
  geom_histogram(bins = 50, fill = "lightgray", color = "black") +
  geom_vline(
    data = df.nested %>% filter(data.group == "obs") %>% left_join(df.results %>% select(trt.set, sig.group)),
    aes(xintercept = diff_value, color = sig.group),
    linewidth = 1
  ) +
  geom_text(
    data = df.results %>% mutate(x = -0.13, y = 55),
    aes(x = x, y = y, label = denote)
  ) +
  theme_bw() +
  xlab("Difference in the relative length\n of the first semi-axis") +
  ylab("Count") +
  theme(
    axis.text = element_text(color = "black"),
    legend.position = "top",
    panel.grid = element_blank()
  ) +
  scale_color_manual(values = c("blue", "black")) +
  labs(color = "Significance:")

p.null
```



2.6 make a figure only showing the main effect of perturbation

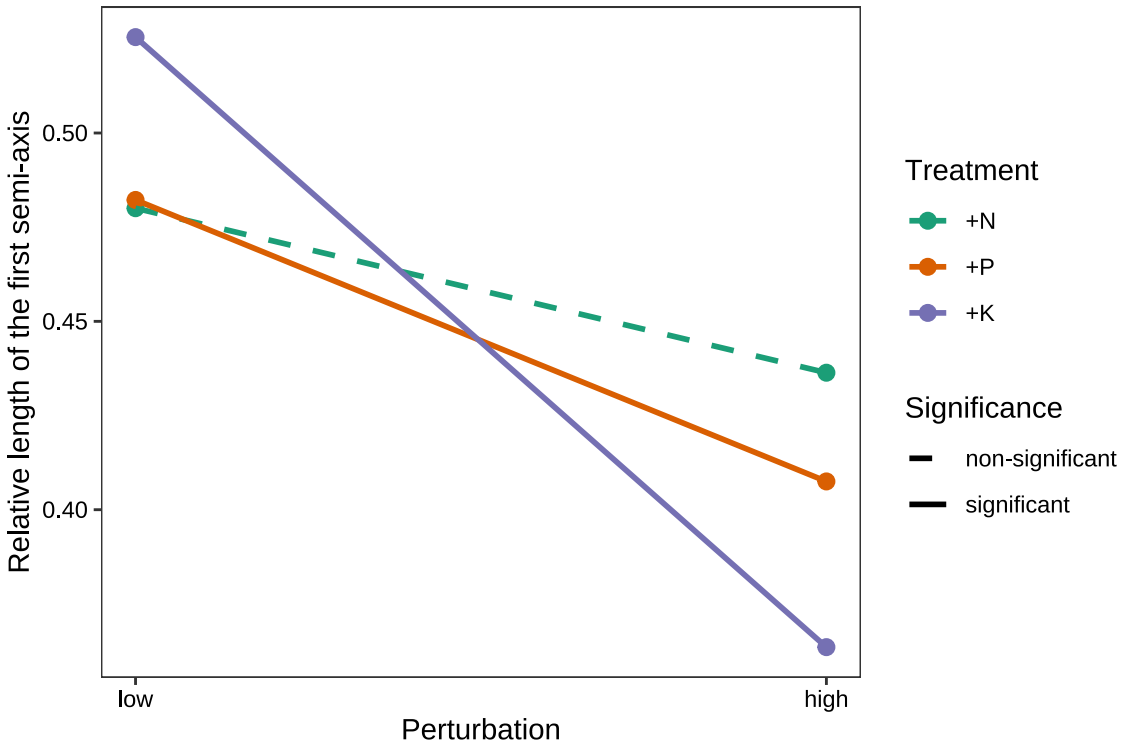
```
df.pert.effect <- df.nested.decoupling.var.explained_func_var_det %>%
  filter(data.group == "obs") %>%
  mutate(
    high = map_dbl(model.results, function(model_) coef(model_)[1]),
    low = map_dbl(model.results, function(model_) coef(model_)[1] + coef(model_)[2])
  ) %>%
  ungroup()
df.pert.effect.long <- df.pert.effect %>%
  rename(Treatment = trt.set) %>%
  select(Treatment, high, low) %>%
  pivot_longer(cols = c(high, low), names_to = "Perturbation", values_to = "Value")
```

```r
df.pvalues <- df.results.decoupling.var.explained_func_var_det %>%
  rename(Treatment = trt.set) %>%
  select(Treatment, denote) %>%
  arrange(desc(Treatment))
df.pvalues$x <- rep(1, 3)
df.pvalues$y <- c(0.38, 0.4, 0.42)

df.pert.effect.long <- df.pert.effect.long %>%
  left_join(df.results.decoupling.var.explained_func_var_det %>%
              rename(Treatment = trt.set) %>% select(Treatment, alpha))
df.pert.effect.long <- df.pert.effect.long %>%
  mutate(Significance = map_chr(alpha, function(x)ifelse(x<0.05|x>0.95, "significant","non-significant")))

P.decoupling.main_func_var_det <- df.pert.effect.long %>%
  mutate(Perturbation = as.factor(Perturbation)) %>%
  mutate(Perturbation = fct_relevel(Perturbation, c("low", "high"))) %>%
  ggplot(aes(x = as.numeric(Perturbation), y = Value, color = Treatment)) +
  geom_point(size = 2.5) +
  geom_line(linewidth = 1, aes(linetype = Significance)) +
  #geom_text(data = df.pvalues, aes(x = x, y = y, label = denote, color = Treatment), hjust = 0, show.legend = F) +
  scale_color_brewer(palette = "Dark2") +
  theme_bw() +
  scale_x_continuous(breaks = c(1, 2), labels = c("low", "high")) +
  xlab("Perturbation") +
  ylab("Relative length of the first semi-axis") +
  theme(axis.text = element_text(color = "black"),
        panel.grid = element_blank())+
  scale_linetype_manual(values = c("dashed","solid"))

P.decoupling.main_func_var_det
```



Section 3: dimensionality analysis using the correlation approach

This analysis will use the same data as that with pca approach.

3.1 compute the correlation coefficient

```
# calculate correlation coefficient
df.permutation %<>% mutate(corr.v = map(data, cor))
df.correlation <- df.permutation %>% select(-data)
```

3.2 GLS models of the difference between nutrient perturbation intensity groups in pairwise correlations among stability responses

```r
# change the correlation matrix to long form and only keep three correlations
df.correlation %<>% mutate(corr.v2 = map(corr.v, function(m) {
  out <- as.data.frame(as.table(m))
  return(out)
}))
df.correlation %<>% select(-corr.v) %>% unnest(corr.v2)

df.correlation <- df.correlation %>%
  mutate(sta_pair = paste(Var1, Var2, sep = " & ")) %>%
  filter(sta_pair %in% c(
    "func_var & comp_var",
    "func_change & comp_change",
    "func_var & func_change",
    "comp_var & comp_change",
    "func_var_det & comp_var",
    "func_var_det & func_change",
    "func_change & comp_var",
    "func_var & comp_change",
    "func_var_det & comp_change"
  )) %>%
  select(-Var1, -Var2) %>%
  rename(Value = Freq) %>%
  mutate(Value.abs = abs(Value))

# factorize sta_pair
df.correlation <- df.correlation %>%
  mutate(sta_pair = str_replace_all(sta_pair, " & ", " &\n ")) %>%
  mutate(sta_pair = as.factor(sta_pair)) %>%
  mutate(sta_pair = fct_relevel(sta_pair, c(
    "func_var &\n comp_var",
    "func_change &\n comp_change",
    "func_var &\n func_change",
    "comp_var &\n comp_change",
    "func_var_det &\n comp_var",
    "func_var_det &\n func_change",
    "func_change &\n comp_var",
    "func_var &\n comp_change",
    "func_var_det &\n comp_change"
```

```
  )))

corrs.all <- df.correlation$sta_pair %>% unique()
corrs.selected <- setdiff(corrs.all, corrs.all[str_detect(corrs.all, "func_var ")])

# nest df.correlation
df.nested <- df.correlation %>%
  select(-final.year.set) %>%
  group_by(trt.set, sta_pair, data.group, perm.id) %>%
  nest()

# perform the gls
# as some of the elements in df.nested will return errors, I here run GLS as a loop
# I also just extract the coefficient in the loop

low.set <- high.set <- results <- numeric(nrow(df.nested))

for (i in 1:nrow(df.nested)) {
  results[i] <- NA
  low.set[i] <- NA
  high.set[i] <- NA

  try({
    dt_ <- df.nested$data[[i]]
    gls_model <- gls(Value.abs ~ PI.group,
      data = dt_,
      correlation = corAR1(form = ~ end_year | PI.group)
    )

    high.set[i] <- coef(gls_model)["(Intercept)"]
    results[i] <- coef(gls_model)["PI.groupLow"] # diff
    low.set[i] <- high.set[i] + results[i]
  })
}

df.nested$diff_value <- results
df.nested$low <- low.set
df.nested$high <- high.set
```

```r
# note that the difference now is calculated as low-perturbation minus high-perturbation,
# I want to use it in an opposite way
df.nested <- df.nested %>%
  mutate(diff_value = -1 * diff_value)
# now factorize trt.set
df.nested <- df.nested %>%
  mutate(trt.set = paste0("+", trt.set)) %>%
  mutate(trt.set = as.factor(trt.set)) %>%
  mutate(trt.set = fct_relevel(trt.set, c("+N", "+P", "+K")))

# only keep the key columns
df.results <- df.nested %>%
  ungroup() %>%
  # select(-data) %>%
  na.omit() %>% # remove the problametic gls result; only one
  group_by(trt.set, sta_pair) %>%
  nest()
# now add a column to represent the significance
df.results <- df.results %>%
  mutate(alpha = map_dbl(data, function(dt_) {
    x <- dt_ %>%
      filter(perm.id != 0) %>%
      select(diff_value) %>%
      unlist()
    x0 <- dt_ %>%
      filter(perm.id == 0) %>%
      select(diff_value) %>%
      unlist()
    length(which(x <= x0)) / length(x)
  }))
# also add a denote colomn for p values so I can add it to the figure later
df.results <- df.results %>%
  mutate(denote = map_chr(alpha, function(x) {
    x <- round(x, digits = 3)
    paste("P =", x)
  }))

df.results <- df.results %>% select(-data) # here
```

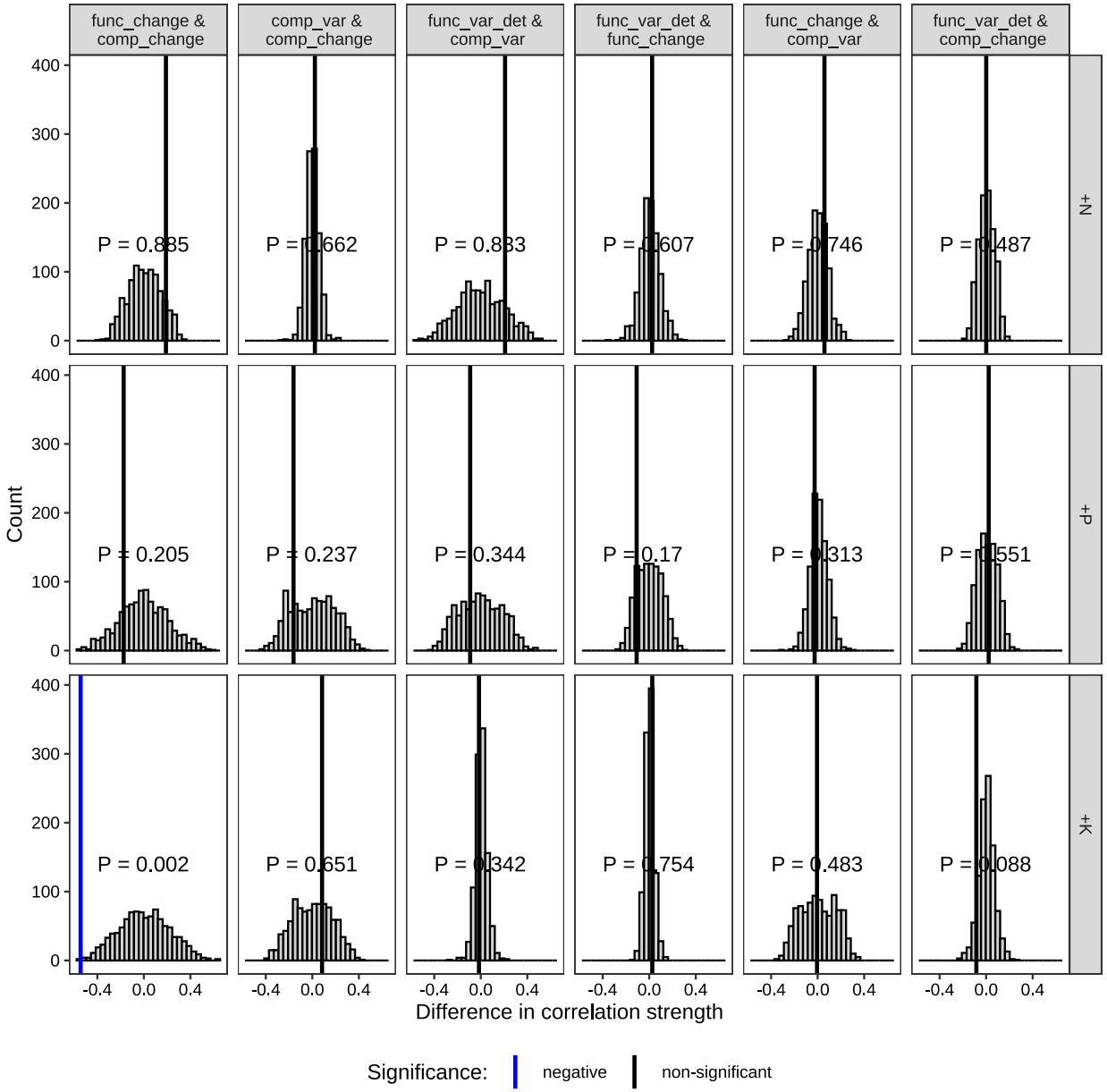
```
# now make a figure for null model result excluding the sta_pair involving func_var

# add a column represent significance groups
df.results <- df.results %>%
  mutate(sig.group = map_chr(alpha, function(x) {
    if (x < 0.05) {
      y <- "negative"
    } else if (x < 0.95) {
      y <- "non-significant"
    } else {
      y <- "positive"
    }
  }))
```

3.3 make a figure of the null distributions of model coefficient estimates for the difference in correlation strength in pairwise correlations between stability components obtained from 1,000 permutations of the data.

```
p.null <- df.nested %>%
  filter(data.group == "permutation", sta_pair %in% corrs.selected) %>%
  ggplot(aes(x = diff_value)) +
  facet_grid(trt.set ~ sta_pair) +
  geom_histogram(bins = 30, fill = "lightgray", color = "black") +
  geom_vline(
    data = df.nested %>% filter(data.group == "obs", sta_pair %in% corrs.selected) %>%
      left_join(df.results %>% select(trt.set, sta_pair, sig.group)),
    aes(xintercept = diff_value, color = sig.group),
    linewidth = 1
  ) +
  geom_text(
    data = df.results %>% mutate(x = -0.4, y = 140) %>% filter(sta_pair %in% corrs.selected),
    aes(x = x, y = y, label = denote),
    hjust = 0
  ) +
  theme_bw() +
  xlab("Difference in correlation strength") +
  ylab("Count") +
  theme(
    axis.text = element_text(color = "black"),
```

```
    legend.position = "bottom",
        panel.grid = element_blank()
  ) +
  scale_color_manual(values = c("blue", "black", "red")) +
  labs(color = "Significance:")
p.null
```


func_change & comp_change
comp_var & comp_change
func_var_det & comp_var
func_var_det & func_change
func_change & comp_var
func_var_det & comp_change
+N
+P
+K
Count
P = 0.885
P = 0.662
P = 0.883
P = 0.607
P = 0.746
P = 0.487
P = 0.205
P = 0.237
P = 0.344
P = 0.17
P = 0.313
P = 0.551
P = 0.002
P = 0.651
P = 0.342
P = 0.754
P = 0.483
P = 0.088
0
100
200
300
400
-0.4
0.0
0.4
Difference in correlation strength
Significance:
negative
non-significant

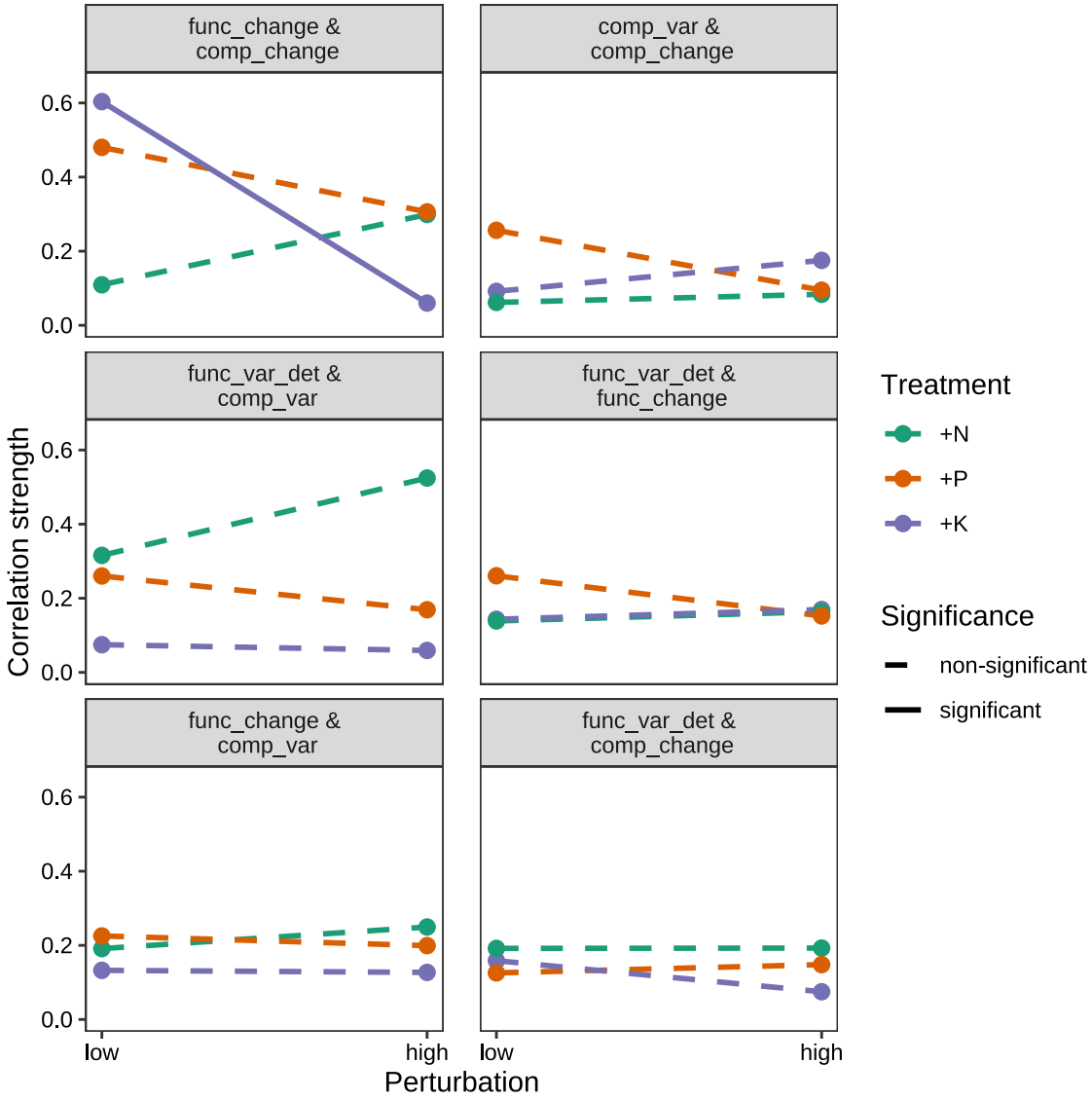
3.4 make a figure for the main effect of perturbation on correlation

```
df.pert.effect <- df.nested %>%
  filter(data.group == "obs") %>%
  ungroup()
df.pert.effect.long <- df.pert.effect %>%
  rename(Treatment = trt.set) %>%
  select(Treatment, sta_pair, high, low) %>%
  pivot_longer(cols = c(high, low), names_to = "Perturbation", values_to = "Value.abs")

df.pert.effect.long <- df.pert.effect.long %>%
  left_join(df.results %>%
              rename(Treatment = trt.set) %>% select(sta_pair, Treatment, alpha))
df.pert.effect.long <- df.pert.effect.long %>%
  mutate(Significance = map_chr(alpha, function(x)ifelse(x<0.05|x>0.95, "significant","non-significant")))

P.corr.main <- df.pert.effect.long %>%
  filter(sta_pair %in% corrs.selected) %>%
  mutate(Perturbation = as.factor(Perturbation)) %>%
  mutate(Perturbation = fct_relevel(Perturbation, c("low", "high"))) %>%
  ggplot(aes(x = as.numeric(Perturbation), y = Value.abs, color = Treatment)) +
  geom_line(linewidth = 1, aes(linetype = Significance)) +
  geom_point(size = 2.5) +
  facet_wrap(~sta_pair, nrow = 3) +
  #geom_text(
  #  data = df.pvalues %>% filter(sta_pair %in% corrs.selected),
   # aes(x = x, y = y, label = denote, color = Treatment), hjust = 0, show.legend = F, size = 3
 # ) +
  scale_color_brewer(palette = "Dark2") +
  theme_bw() +
  scale_x_continuous(breaks = c(1, 2), labels = c("low", "high")) +
  xlab("Perturbation") +
  ylab("Correlation strength") +
  theme(
    axis.text = element_text(color = "black"),
    panel.spacing.x = unit(.5, "cm"),
        panel.grid = element_blank()
  )+
  scale_linetype_manual(values = c("dashed","solid"))+
```

```
  ylim(c(0,0.65))
P.corr.main
```


func_change & comp_change
comp_var & comp_change
func_var_det & comp_var
func_var_det & func_change
func_change & comp_var
func_var_det & comp_change
Correlation strength
0.0
0.2
0.4
0.6
low
high
Perturbation
Treatment
+N
+P
+K
Significance
non-significant
significant